

Program Information:

I have uploaded my C++ project to the GitHub for this class (https://github.com/tgmaxson/HPC_TGM) under folder “hw1”. The program uses dynamic arrays and simple functions to iterate over cells and follow the Game of Life logic. Compiling is done by running the compile-all.sh script, which is copied below.

```
#!/bin/bash

mkdir bin

gcc -lm -Wall -O0 -o bin/hw1-gcc-O0 hw1.c
gcc -lm -Wall -O3 -o bin/hw1-gcc-O3 hw1.c
icc -lm -Wall -O3 -o bin/hw1-icc-O3 hw1.c
```

Cluster Information:

I am running on Ubuntu 22.04.2 LTS via a cluster known as “Clustie” to our research group, which I have full access to. Tests are run on a test node which has nothing else running at the time of execution, except for SLURM and other background tasks which are default to Ubuntu Server installs. GCC is version 11.4.0 and ICC is version 2021.10.0 via OneAPI released on 06/09/2023. CUDA is provided via Nvidia HPC SDK version and nvcc is version *cuda_12.0.r12.0/compiler.32267302_0* with CUDA version V12.0.140.

System Hardware:

Motherboard: H11SSL-i

CPU: Single Socket 32 Core AMD Epyc 7551P, 2.0 GHz, boost to 3.0 GHz. SMT Disabled.

Memory: 256 GB of ECC DDR4-2400 via 8 x 32 GB sticks in 8 channel

GPU: 2 x MSI Suprim Liquid X24G RTX 4090 with 24GB of memory.

Profiling:

For this test case, profiling is done via a simple “time” command tracking time internal to the program as we do not need information about specific regions of the program, just total execution time. Here we profile GCC and ICC at -O3 optimizations. We take our start time right at the program start and end time right as the last print statement is called. This may exclude some startup time at the OS level, but this is negligible and not something we are interested in right now. Values are averaged over 3 runs currently to account for different random starts, but more runs would be better for a real problem. Since I did not run this on a local machine (as HW1 seems to expect), I have included -O0 in my list of tested executables.

Profiling is performed using the Slurm script “profile.sh” which is provided in the Git repository.

Test Case #	Problem Size (N x N)	Max Generations	GCC -O3	ICC -O3	GCC -O0
-------------	-------------------------	--------------------	---------	---------	---------

			Time Taken (Seconds)	Time Taken (Seconds)	Time Taken (Seconds)
1	1000	1000	1.378	1.340	33.268
2	1000	5000	6.662	6.674	166.945
3	5000	1000	49.576	55.918	791.073
4	5000	5000	225.132	243.204	4074.818
5	10000	1000	185.856	206.668	3479.371
6	10000	5000	859.908	931.664	17063.785

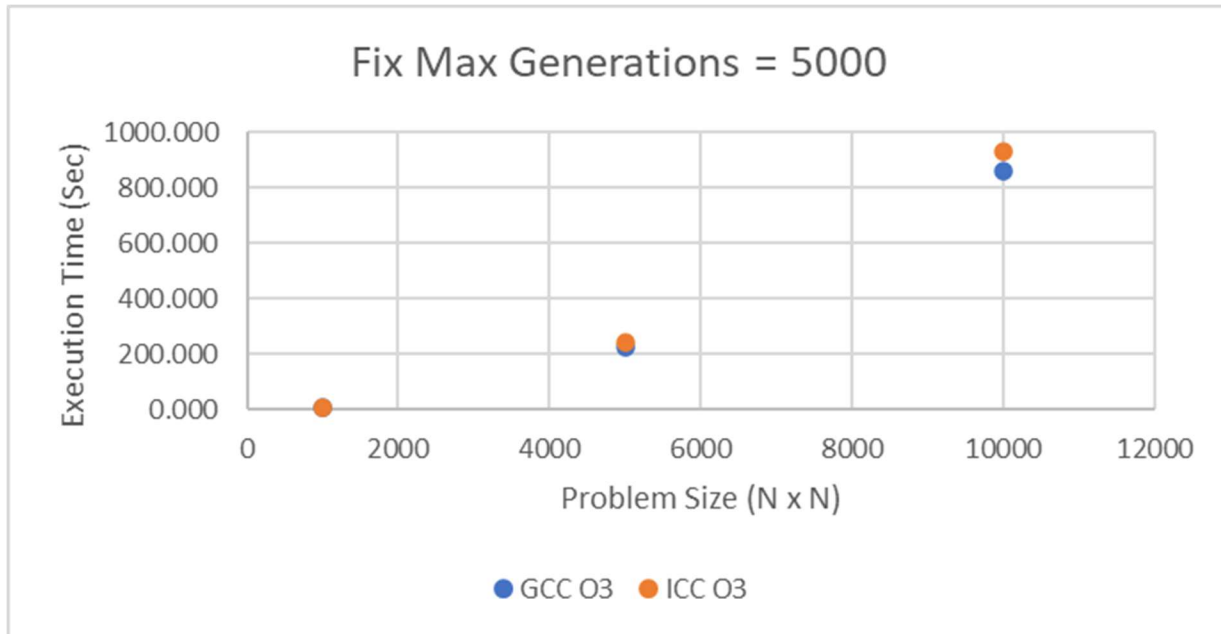


Figure 1: Scaling of performance with regard to the problem size for 5000 generations. GCC O0 is excluded as it performs much worse than O3 for GCC and ICC and is not competitive in speed.

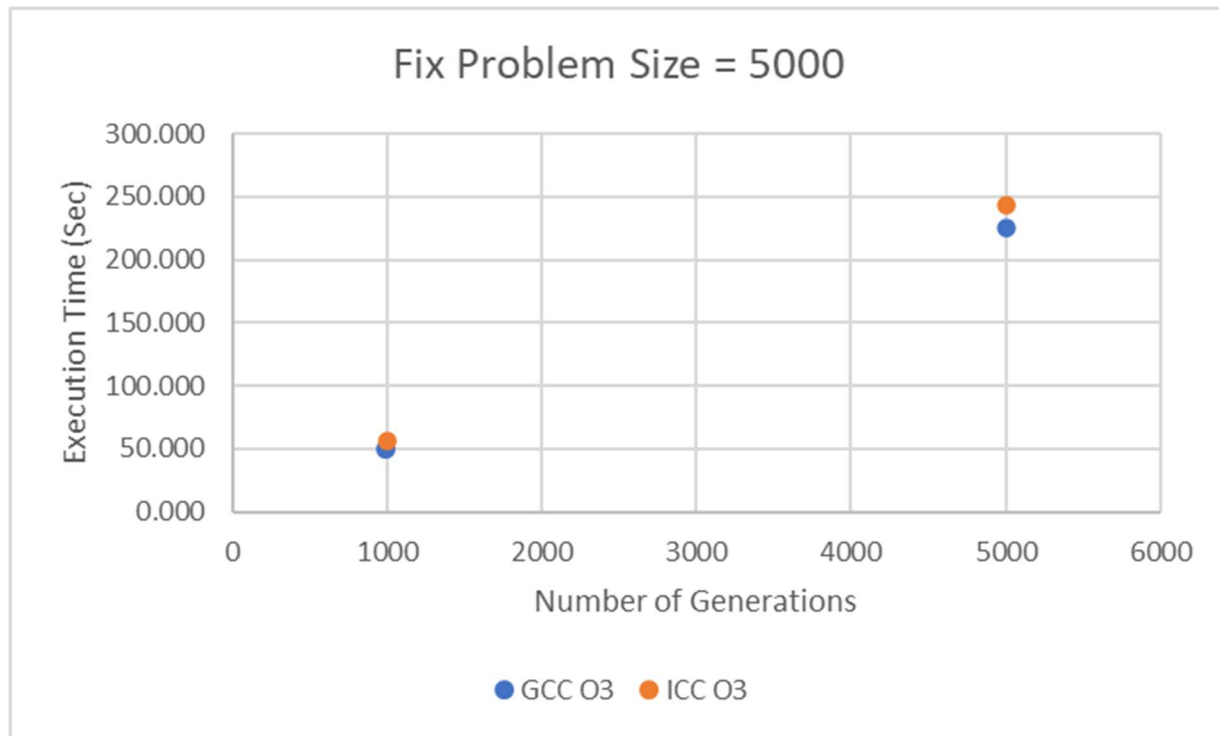


Figure 2: Scaling of performance with regard to the number of generations for a fixed problem size of $N=5000$ which gives a 5000×5000 game board. GCC O0 is excluded as it performs much worse than O3 for GCC and ICC and is not competitive in speed.

Notable in the performance data is some interesting trends which can be seen by looking at Figure 1 and Figure 2. Going from 1,000 to 5,000 as a problem size results in a 25X increase in cells (5×5) but a ~50X increase in time it takes to solve the problem. When going to a problem size of 10,000 from 5,000, the problem increases by 4X and an ~4X increase in time is observed. This indicates that at 5,000 problem size, the arrays likely no longer fit in some cache and performance decreases as a result by 2X more than it should if the scaling was linear with problem size. However, 5,000 to 10,000 shows a nearly linear scaling. The performance change as the number of generations is changed is almost exactly linear and this makes sense as the number of operations required to execute should not change as the program progresses. The exception to this would be that the random start would make the starting configurations more unstable (and thus branch differently), but after a few iterations it should be mostly stable configurations in the game of life.

It appears ICC is about 6% slower than GCC over all these tests at optimization level O3. This is not terribly surprising as the processors are AMD based, but still somewhat surprising as real research programs we use still tend to prefer ICC for better performance. It may be the lack of MKL or MPI that results in us seeing this difference in this program where GCC is favored. It also appears that ICC does less well for larger problems than GCC. The use of -O0 seems to result in a 30X decrease in speed, indicating how important compiler optimizations are in this program.

Technical Paper:

What is Moore's Law?

Moore's law is the observation / prediction that the number of transistors will double in an integrated circuit every two years for a given price point. This implies computing power doubles every two years as well.

What is Dennard scaling?

As the voltage is reduced to a component, the power used is also reduced. This allows for the same amount of power to be used to power more transistors while producing the same level of heat.

What is S-curve model?

An s-curve model is an alternative to an exponentially growing model. A time of rapid growth is followed by a plateau at some point, resembling an S shape.

Is Moore's Law dead now?

In the traditional sense, yes as transistor density is only slowly increasing. However, computing power is still effectively maintaining Moore's Law via improvements in chip design, multicore performance, and software improvements. For this reason, it can be said that Moore's Law is still valid and is seen as a target for CPU engineering to maintain as long as possible.

According to the authors, what are the three levels of exponential growth in the computing ecosystem? Will this growth be sustained for the next decade?

The three levels would be chip, computer, and adopting community. The authors believe that this type of growth described by Moore's Law will be sustained and exponential growth of some kind will be sustained much longer.